There are a couple of opportunities for deadlock here, because you call user code while holding a lock: the copy constructor or move constructor ❶, ❸ and copy assignment or move assignment operator ❺ on the contained data items, as well as potentially a user-defined `operator new`. If these functions either call member functions on the stack that the item is being inserted into or removed from or require a lock of any kind and another lock was held when the stack member function was invoked, there's the possibility of deadlock. However, it's sensible to require that users of the stack be responsible for ensuring this; you can't reasonably expect to add an item onto a stack or remove it from a stack without copying it or allocating memory for it.

Because all the member functions use a `std::lock_guard<>` to protect the data, it's safe for any number of threads to call the `stack` member functions. The only member functions that aren't safe are the constructors and destructors, but this isn't a particular problem; the object can be constructed only once and destroyed only once. Calling member functions on an incompletely constructed object or a partially destructed object is never a good idea whether done concurrently or not. As a consequence, the user must ensure that other threads aren't able to access the stack until it's fully constructed and must ensure that all threads have ceased accessing the stack before it's destroyed.

Although it's safe for multiple threads to call the member functions concurrently, because of the use of locks, only one thread is ever actually doing any work in the stack data structure at a time. This *serialization* of threads can potentially limit the performance of an application where there's significant contention on the `stack`: while a thread is waiting for the lock, it isn't doing any useful work. Also, the stack doesn't provide any means for waiting for an item to be added, so if a thread needs to wait, it must periodically call `empty()` or just call `pop()` and catch the `empty_stack` exceptions. This makes this stack implementation a poor choice if such a scenario is required, because a waiting thread must either consume precious resources checking for data or the user must write external wait and notification code (for example, using condition variables), which might render the internal locking unnecessary and therefore wasteful.